Documentação do Playsaurus

Documentação do Playsaurus

PARA DESENVOLVEDORES

Arquitetura do Playsaurus

O Playsaurus é uma ferramenta local para manter a documentação de vários produtos a partir de uma única instalação. Cada produto possui seus próprios documentos, tema, configuração, screenshots e artefatos; o código da ferramenta permanece compartilhado. Stack

PARA USUÁRIOS

Usando o Playsaurus

O painel do Playsaurus concentra o fluxo diário: escolher um projeto, conferir onde está o repositório na máquina, gerar a documentação, visualizar as duas versões e copiar o resultado para o produto. Escolher um projeto No combo Projeto , selecione o produto

CONSULTA RÁPIDA

Referência do Playsaurus

Consulta rápida dos comandos, arquivos e convenções usados pela ferramenta. Comandos principais Comando Função --- --- npm run painel Abre o painel local npm run build -- Gera os pacotes interno e cliente npm run start -- Gera e serve a documentação interna np

Arquitetura do Playsaurus

O Playsaurus é uma ferramenta local para manter a documentação de vários produtos a partir de uma única instalação. Cada produto possui seus próprios documentos, tema, configuração, screenshots e artefatos; o código da ferramenta permanece compartilhado.

Stack principal

Camada	Tecnologia	Função
Runtime	Node.js	Painel local, build, publicação e automações
Conteúdo	Markdown	Fonte dos artigos da documentação
Renderização	markdown-it	Conversão do Markdown para HTML
Automação de navegador	Playwright	Screenshots e fluxos auxiliares
Diagramas	Mermaid	Diagramas incorporados aos artigos
PDF	Playwright + pdf-lib	Impressão e consolidação da documentação
Configuração	JSON, CSS e `.env`	Projeto, tema e credenciais locais

O portal publicado é standalone: o produto cliente recebe arquivos estáticos e não precisa executar o Playsaurus em produção.

Organização da instalação

```
playsaurus/
├── painel/                 interface local
├── scripts/                build, preview, PDF e publicação
├── compartilhado/          compilador e runtime do portal
├── extras/                 auditoria, vídeo e auxiliares opcionais
```

```
└── projetos/
    └── <id>/
        ├── projeto.json
        ├── tema.css
        ├── docs/
        ├── i18n/
        ├── static/
        ├── playwright/
        ├── output/
        └── .playsaurus/
```

O conteúdo permanente de um produto fica em `projetos/<id>/`. `output/` e `.playsaurus/` são regeneráveis.

Modelo de páginas

Durante o build, os arquivos `.md` são lidos e normalizados em um único modelo de páginas. Esse mesmo modelo alimenta:

- o portal standalone;
- `help-index.json`;
- `sitemap.xml` do pacote cliente;
- navegação e busca;
- exportação para PDF.

Isso evita que cada saída interprete a documentação de uma forma diferente.

Build interno e cliente

Um único comando gera os dois artefatos:

```
npm run build -- <id>
```

Saídas:

```
projetos/<id>/output/
├── interno/
```

```
└── cliente/
```

O pacote **interno** pode conter todas as seções. O pacote **cliente** remove as seções marcadas com `"publicar": false` em `projeto.json`.

Essa separação é feita durante o empacotamento. Um link de uma página pública para uma seção interna é mantido como texto, mas a navegação é desativada no pacote cliente.

Publicação

`npm run publish -- <id>` gera a documentação e copia somente `output/cliente/` para o caminho configurado em `repositorio.destino`, normalmente `public/docs` do produto.

O caminho local do repositório não precisa ficar gravado no projeto: o painel salva caminhos específicos da máquina em `caminhos.local.json`.

Dados sensíveis

Credenciais usadas por screenshots ou auditorias ficam em `projetos/<id>/.env`, que não deve ser versionado. O `projeto.json` contém apenas nomes de variáveis e configuração não sensível.

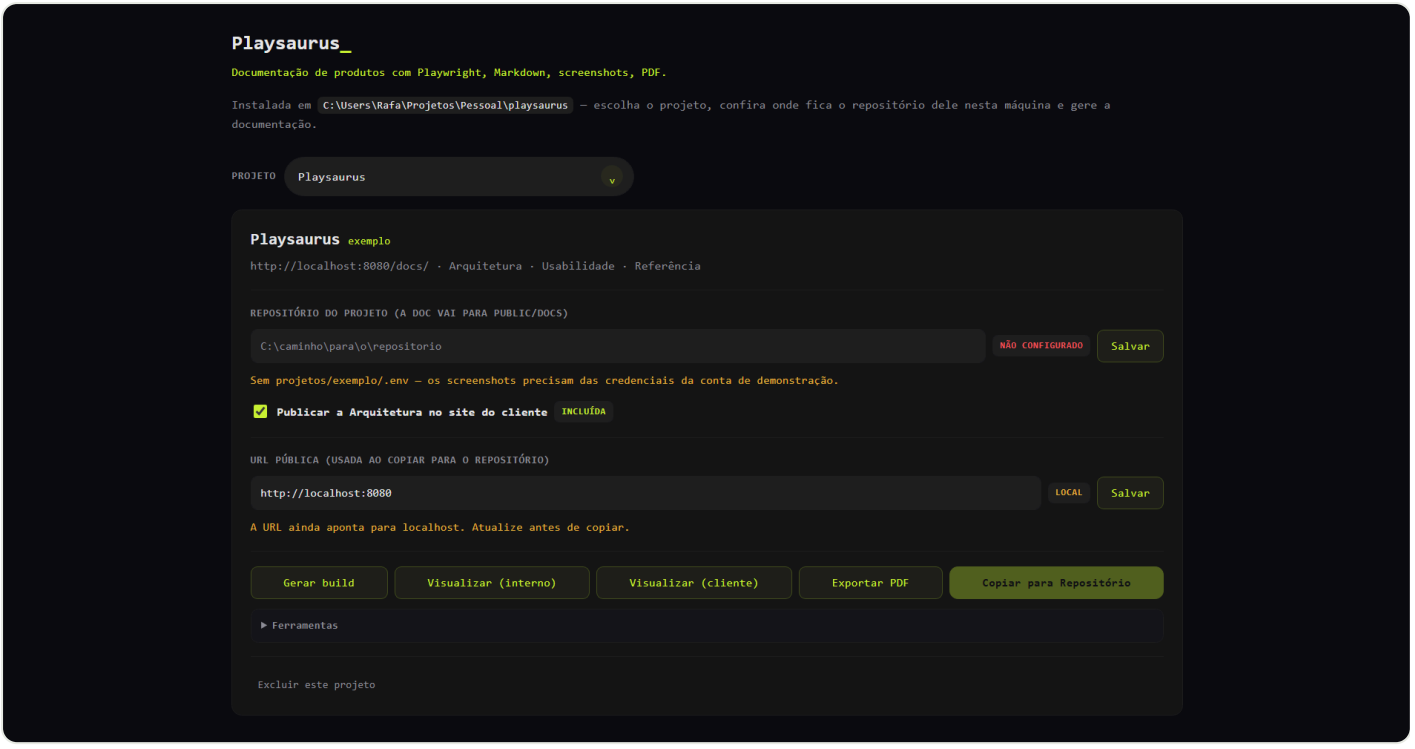
A opção `publico: admin` de um artigo serve para filtragem da Central de Ajuda; ela não substitui a separação entre conteúdo interno e cliente.

Extras

Auditoria e gravação de vídeo não participam do build normal. Elas ficam em `extras/` e são executadas somente quando chamadas pelo CLI, mantendo o núcleo da documentação menor e mais previsível.

Usando o Playsaurus

O painel do Playsaurus concentra o fluxo diário: escolher um projeto, conferir onde está o repositório na máquina, gerar a documentação, visualizar as duas versões e copiar o resultado para o produto.



Escolher um projeto

No combo **Projeto**, selecione o produto que deseja documentar. O cartão exibe o endereço da documentação, as seções disponíveis e a configuração local do repositório.

Se o caminho do repositório ainda não estiver configurado nesta máquina, informe-o e clique em **Salvar**. As ações que dependem desse caminho permanecem bloqueadas até a configuração ser salva.

Configurar o que vai para o cliente

Quando o projeto possui uma seção **Arquitetura**, o painel permite definir se ela entra ou não no site do cliente.

- **Incluída:** Arquitetura aparece no build cliente.
- **Só interna:** Arquitetura existe apenas no build interno.

Essa configuração altera o conteúdo publicado; não é apenas uma opção visual do painel.

URL pública

A **URL pública** é usada pelo sitemap e pelos links absolutos do pacote publicado. Antes de copiar a documentação para um ambiente real, troque URLs de `localhost` pelo endereço definitivo e salve.

Gerar a documentação

Clique em **Gerar build** para produzir:

```
output/interno/
output/cliente/
```

O build lê Markdown, incorpora os assets necessários e atualiza os índices auxiliares.

Visualizar

Use:

- **Visualizar (interno)** para conferir todas as seções;
- **Visualizar (cliente)** para conferir exatamente o que será copiado para o produto.

Antes de publicar, a visualização cliente é a referência para verificar se conteúdo interno foi realmente removido.

Exportar PDF

Exportar PDF gera a documentação em A4 usando o próprio portal final como fonte. Assim, o PDF segue as mesmas páginas e os mesmos idiomas do build standalone.

Quando existe um único idioma, os arquivos usam nomes simples como:

```
documentacao-cliente.pdf
documentacao-equipe.pdf
```

Copiar para Repositório

Depois de revisar o build cliente, clique em **Copiar para Repositório**. O Playsaurus copia o conteúdo de `output/cliente/` para o destino configurado no repositório do produto.

Esse passo não envia código-fonte do Playsaurus nem os documentos Markdown; apenas o artefato final é copiado.

Ferramentas

A área **Ferramentas** reúne ações usadas com menos frequência, como:

- gerar screenshots;
- abrir a pasta do projeto;
- abrir a pasta de saída;
- limpar cache;
- limpar builds.

Screenshots podem exigir credenciais de demonstração em `projetos/<id>/.env`. O painel mostra um aviso quando elas ainda não estão configuradas.

Criar um novo projeto

Na área **Novo projeto**, preencha:

1. **Identificador** — vira o nome da pasta e o argumento dos comandos;
2. **Nome do projeto** — nome exibido no portal;
3. **Endereço do app** — URL usada para screenshots;
4. **Caminho da documentação** — normalmente `/docs/`;
5. **Idioma padrão**;
6. **Cor da marca**.

O Playsaurus cria o esqueleto inicial em `projetos/<id>/`, incluindo as três seções padrão, tema e arquivos de configuração.

Referência do Playsaurus

Consulta rápida dos comandos, arquivos e convenções usados pela ferramenta.

Comandos principais

Comando	Função
`npm run painel`	Abre o painel local
`npm run build -- <id>`	Gera os pacotes interno e cliente
`npm run start -- <id>`	Gera e serve a documentação interna
`npm run serve -- <id>`	Serve um build já existente
`npm run screenshots -- <id>`	Captura screenshots do produto
`npm run pdf -- <id>`	Gera PDFs cliente/equipe
`npm run publish -- <id>`	Copia o pacote cliente para o repositório do produto
`npm run i18n:init -- <id>`	Cria scaffolds de tradução
`npm run i18n:check -- <id>`	Verifica traduções pendentes
`npm run auditoria -- <id>`	Executa a auditoria opcional
`npm run videos -- <id>`	Executa a gravação opcional de tutorial

Arquivos do projeto

Arquivo/pasta	Uso
`projeto.json`	Nome, URLs, seções, idiomas e publicação
`tema.css`	Tokens visuais do portal

Arquivo/pasta	Uso
`docs/`	Markdown no idioma padrão
`i18n/<locale>/`	Traduções
`static/`	Imagens e outros assets fonte
`playwright/`	Login e specs de screenshots
`.env`	Credenciais locais; não versionar
`output/interno/`	Portal completo
`output/cliente/`	Portal pronto para publicação
`.playsaurus/`	Cache e temporários descartáveis

Arquivos gerados

Um pacote cliente pode conter:

```
index.html
help-index.json
sitemap.xml
pdf/
```

`index.html`

Portal standalone com conteúdo, navegação, busca, tema, idiomas e assets incorporados.

`help-index.json`

Índice estruturado das páginas e FAQs. Pode ser consumido por uma Central de Ajuda ou outra integração do produto.

`sitemap.xml`

Gerado somente para o pacote cliente quando existe URL pública. Usa as URLs canônicas do portal, por exemplo:

```
https://produto.exemplo/docs
https://produto.exemplo/docs/usabilidade
https://produto.exemplo/docs/usabilidade/login
```

O sitemap não publica caminhos físicos como `index.html`.

Convenções de documentação

Página inicial da seção

```
docs/usabilidade/index.md
```

vira:

```
/docs/usabilidade
```

Ordem das páginas

Use prefixos numéricos:

```
01-primeiros-passos.md
02-login.md
10-configuracoes.md
```

O número define a ordem, mas não entra na URL.

Recursos Markdown suportados

- headings e âncoras;
- links relativos;
- imagens;
- tabelas;

- blocos de código;
- Mermaid;
- admonitions `:::info`, `:::warning`, `:::danger` e equivalentes.

Seções padrão

Seção	Objetivo
Arquitetura	Informação técnica e interna
Usabilidade	Guias orientados às tarefas do usuário
Referência	Consulta rápida, regras, glossário e FAQ

A visibilidade do cliente é controlada por seção no `projeto.json`/painel.

Status comuns do painel

- **Configurado aqui** — o caminho local do repositório foi salvo nesta máquina;
- **Não configurado** — é necessário salvar o caminho antes das ações dependentes dele;
- **Local** — a URL pública ainda aponta para localhost;
- **Só interna** — a seção não entra no pacote cliente;
- **Incluída** — a seção entra no pacote cliente.